

PROJECT DOCUMENTATION REPORT

ON

**EMPLOYEE RAISE ISSUE
MANAGEMENT**

Record Producer & Employee Center Integration

A ServiceNow Application for Centralized Employee Issue Submission

Submitted by

KOWSHIK NAIDU VALISETTY

August 2026

TABLE OF CONTENTS

1. Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Proposed Solution	4
2. Project Objectives	5
3. Key Features	6
4. ServiceNow Developer Setup	7
4.1 ServiceNow Developer Access	7
4.2 Personal Developer Instance	8
4.3 Development Environment	9
5. Project Implementation in ServiceNow	10
5.1 Custom Application Creation	10
5.2 Employee Issue Custom Table	12
5.3 Raise Employee Issue Record Producer	14
5.4 Record Producer Variables	17
5.5 Category and Subcategory Configuration	19
5.6 UI Policies	21
5.7 Catalog Client Script	24
5.8 Form Validation	27
5.9 Employee Center	28
5.10 Custom Widgets	30
5.11 End-to-End Workflow	31
5.12 Final Application Configuration	32
6. Testing and Output	33
6.1 Test Environment	34
6.2 Test Scenarios	34
6.3 Output Screenshots	35
7. Conclusion	36

1. INTRODUCTION

1.1 Background

In large enterprise organizations, employees regularly encounter workplace-related issues such as hardware problems, access-related issues, infrastructure requests, payroll discrepancies, and HR-related clarifications. These issues require a structured mechanism through which employees can submit the required information, and the responsible teams can manage the resulting requests.

Traditional approaches such as email and informal communication can make issue information inconsistent and may require additional manual effort to organize and manage requests. A centralized service-management platform can provide a more structured approach by allowing employees to submit requests through a standardized interface and by maintaining the submitted information as manageable records.

ServiceNow provides platform capabilities for creating customized applications, tables, forms, Record Producers, Service Portal experiences, validation controls, and other configuration components. This project uses these capabilities to develop an **Employee Raise Issue** solution.

The application provides an employee-facing submission process through the ServiceNow Service Portal. A **Record Producer** is used to capture issue information and create a corresponding record in the **Employee Issue custom table**.

The implemented solution focuses on providing a structured submission process while using configured field behavior, validation, and Category–Subcategory dependency to improve the consistency of the information captured.

1.2 Problem Statement

Employees need a simple and standardized mechanism to raise workplace-related issues with complete and relevant information. When issues are raised through emails or informal communication channels, organizations may experience:

- Lack of standardized data capture.
- Delayed issue resolution.
- Limited visibility into issue status.

- Increased manual effort for HR and support teams.
- Inconsistent prioritization and assignment.
- Difficulty in classifying issues consistently.

Therefore, there is a need for a centralized solution that allows employees to raise workplace-related issues through a structured interface while ensuring that relevant information is captured and stored as manageable records.

1.3 Proposed Solution

The proposed solution is a **Raise Employee Issue Record Producer integrated with the ServiceNow Employee Center**.

The employee accesses the Employee Raise Issue service through the Employee Center and submits the required information using the Record Producer. The submission captures information such as:

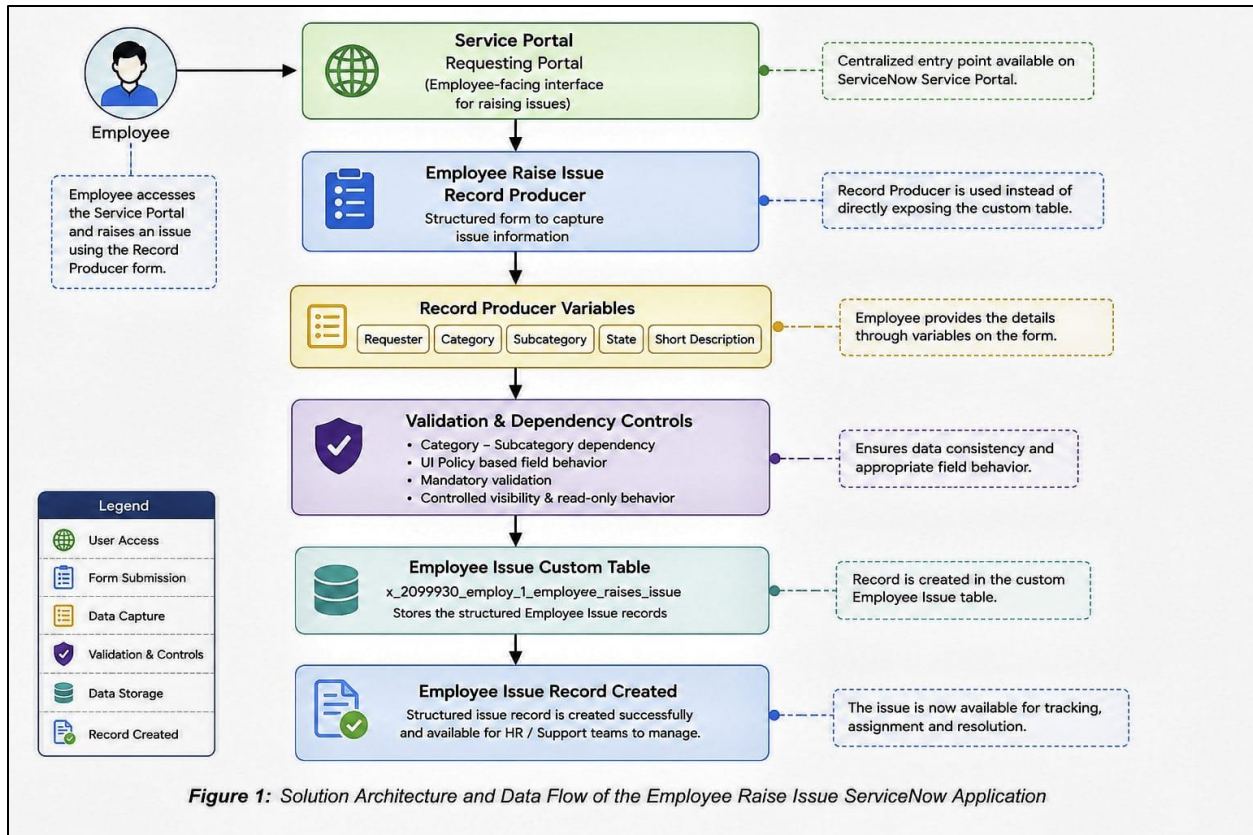
- Requester
- Category
- Subcategory
- State
- Short Description

Configured validation and field dependencies control the submission process. The submitted Record Producer variables are mapped to the corresponding fields of the Employee Issue custom table.

After successful submission, a structured **Employee Issue record** is created in ServiceNow. The resulting record can then be accessed and managed within the application.

The overall solution flow is:

Employee → Service Portal → Record Producer → Issue Information → Validation & Dependencies → Employee Issue Custom Table → Employee Issue Record



2. PROJECT OBJECTIVES

The main objective of the project is to develop a centralized ServiceNow-based mechanism that enables employees to raise workplace-related issues through a structured and controlled submission process.

The specific objectives are:

1. To provide employees with an easy-to-use issue submission interface through the ServiceNow Employee Center.
2. To implement a **Raise Employee Issue Record Producer** for creating Employee Issue records.
3. To capture employee issue information in a standardized format using configured Record Producer variables.
4. To create and use an **Employee Issue custom table** for storing submitted issue information.

5. To implement **Category and Subcategory** classification for workplace-related issues.
6. To configure field dependencies so that Subcategory options respond appropriately to the selected Category.
7. To implement validation and controlled field behavior using ServiceNow configuration capabilities.
8. To map Record Producer variables to the corresponding Employee Issue table fields.
9. To verify that valid submissions successfully create Employee Issue records.
10. To provide a foundation that can be extended with additional employee-service functionality and management capabilities.

3. KEY FEATURES

The Employee Raise Issue application includes the following key features:

3.1 Service Center Access

Employees can access the Employee Raise Issue service through the configured ServiceNow Employee Center.

3.2 Record Producer-Based Submission

A Record Producer is used instead of exposing the Employee Issue custom table directly to employees. This provides a controlled and user-friendly submission experience.

3.3 Structured Issue Information

The submission interface captures relevant information including Requester, Category, Subcategory, State, and Short Description.

3.4 Category–Subcategory Dependency

Subcategory options are controlled according to the selected Category, helping maintain consistent issue classification.

3.5 Validation and Field Behavior

Configured UI Policies and validation controls are used to control field behavior and ensure that required information is captured according to the configured conditions.

3.6 Record Producer Variable Mapping

Values entered through the Record Producer are mapped to the corresponding fields in the Employee Issue custom table.

3.7 Employee Issue Record Creation

After a successful submission, ServiceNow creates a structured Employee Issue record that can be accessed and managed within the application.

3.8 Issue Management

The created Employee Issue records provide a structured representation of submitted workplace issues, supporting subsequent tracking and management.

4. SERVICENOW DEVELOPER SETUP

4.1 ServiceNow Developer Access

ServiceNow development requires access to a ServiceNow development environment in which applications and platform configurations can be created and tested.

For this project, the ServiceNow development environment was used to configure the Employee Raise Issue application and its associated platform components.

The development environment provided access to the required ServiceNow application development capabilities, including table configuration, Record Producers, catalog variables, UI Policies, Catalog Client Scripts, Service Portal configuration, and application testing.

4.2 Personal Developer Instance

A **Personal Developer Instance (PDI)** was used as the development and testing environment for the project.

The PDI provided an isolated ServiceNow instance in which the Employee Raise Issue application could be configured and tested without affecting a production environment.

The project configuration and testing were performed within the PDI, including:

- Creation and configuration of the Employee Issue custom table.
- Configuration of the Raise Employee Issue Record Producer.
- Creation of Record Producer variables.
- Configuration of Category and Subcategory values.
- Implementation of UI Policies and validation behavior.
- Configuration of the Category–Subcategory dependency.
- Testing of Employee Issue record creation.
- Verification of the Service Portal submission process.

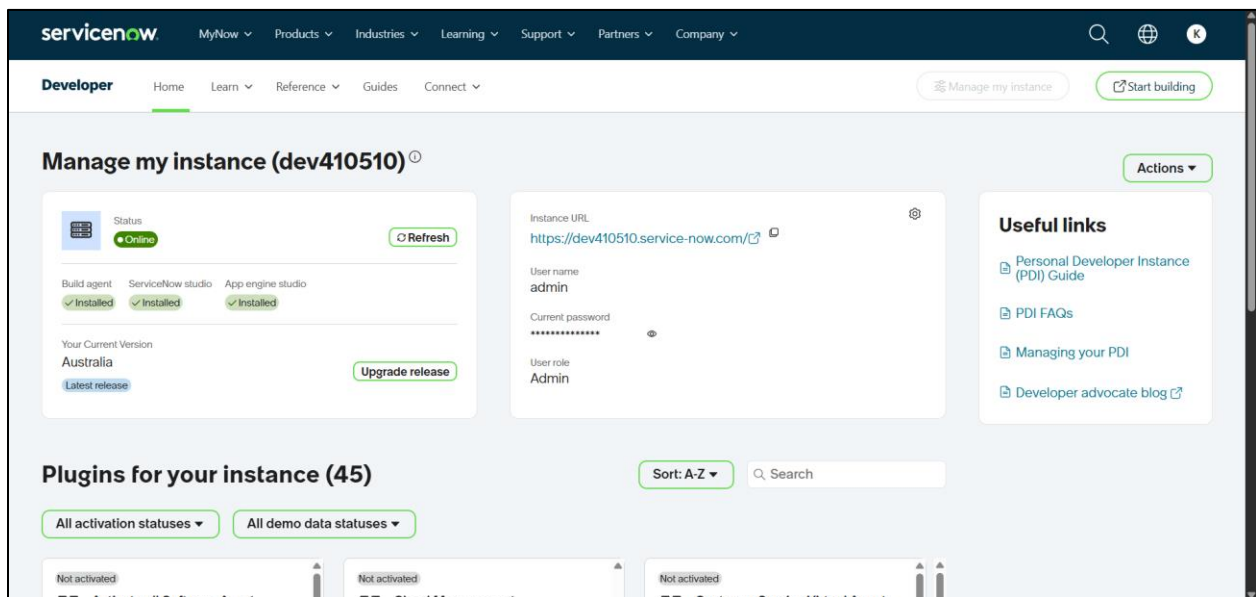


Figure 2: ServiceNow Personal Developer Instance Used for Project Development

4.3 Development Environment

The project was developed and tested using the ServiceNow platform's native application-development and configuration capabilities. The major development components used in the environment were:

Component	Purpose
ServiceNow Platform	Application development and configuration
Employee Issue Custom Table	Storage of submitted employee issues
Record Producer	Employee-facing issue submission
Record Producer Variables	Capture issue information
UI Policies	Control field behavior and validation
Catalog Client Script	Implement Category–Subcategory dependency
Employee Center	Employee-facing access to the issue submission service
ServiceNow Data Layer	Storage and retrieval of Employee Issue records

The development environment supported the complete flow from employee issue submission to creation of a structured Employee Issue record.

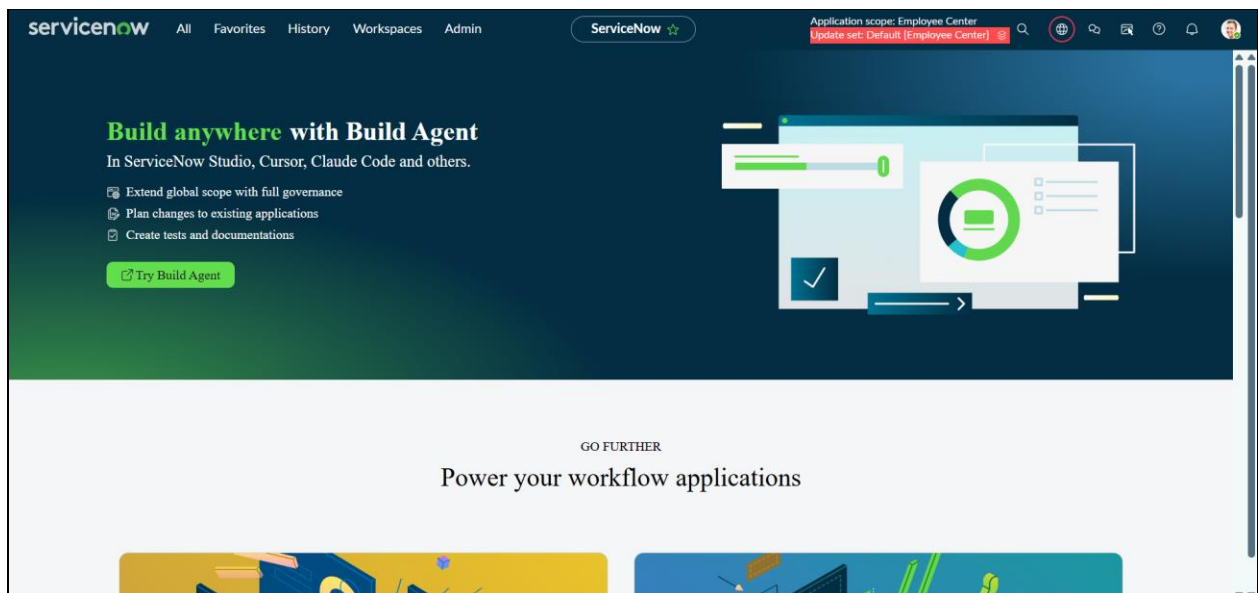


Figure 3: ServiceNow Developer Environment

End-to-End Development Flow

Service Portal → Record Producer → Variables → Validation & Dependency Controls → Employee Issue Custom Table → Employee Issue Record

5. PROJECT IMPLEMENTATION IN SERVICENOW

5.1 Custom Application Creation

The Employee Raise Issue solution was developed within the custom ServiceNow application named Employee Center. The application provides the scope in which the Employee Issue table and the associated configuration components are developed and managed.

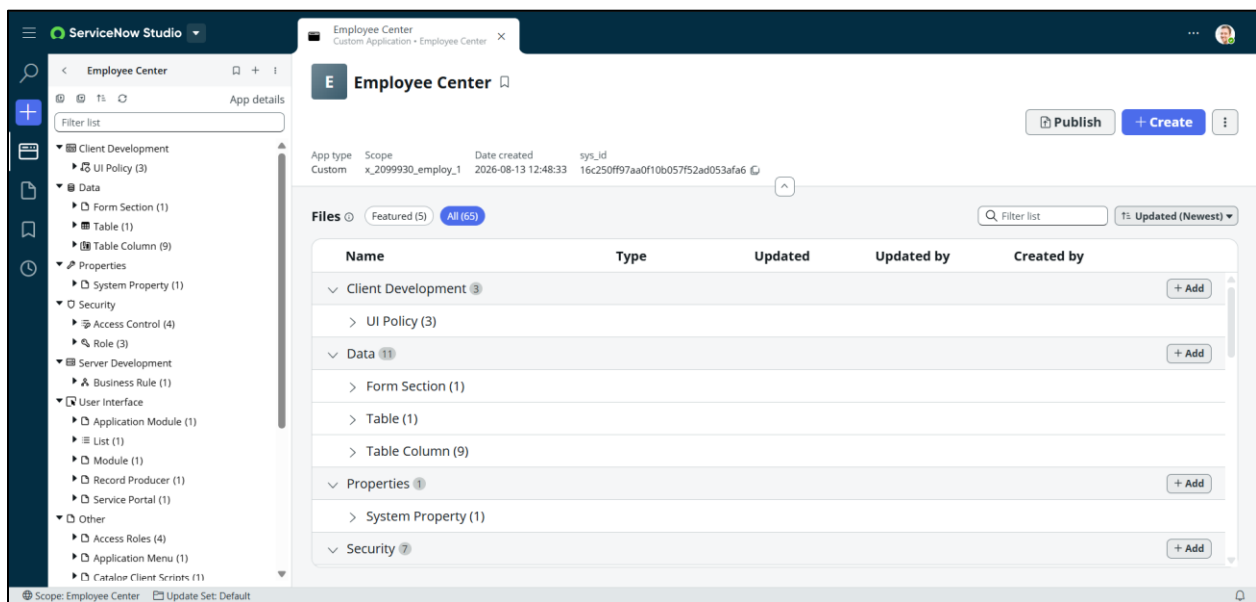


Figure 4: *Employee Center Custom Application and Project Configuration Components*

The custom application serves as the foundation for the Employee Raise Issue solution. The application configuration supports the implementation of the Employee Issue custom table, Record Producer, variables, UI Policies, validation, Category–Subcategory dependency, and Service Portal submission experience.

The application-based approach keeps the project components organized within the ServiceNow development environment and provides a structured foundation for further configuration.

Application Configuration

The application was configured as the development container for the Employee Raise Issue solution.

Configuration	Details
Application Type	Custom ServiceNow Application (Employee Center)
Application Purpose	Employee Raise Issue management
Primary Function	Provide a structured mechanism for employees to raise workplace-related issues
Main Data Object	Employee Issue custom table
User Interface	Employee Center
Submission Mechanism	Raise Employee Issue Record Producer
Application Components	Custom Table, Record Producer, Variables, UI Policies, Client-side configuration

The application provides the foundation for the subsequent implementation of the Employee Issue table and employee-facing issue submission process.

Application Scope

The application scope is used to organize the project-specific configuration within the ServiceNow development environment. The Employee Raise Issue components are developed within the application's context so that the related configuration can be maintained as part of the solution.

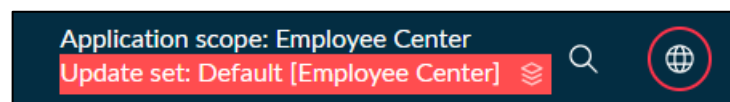


Figure 5: *Employee Center Application Scope*

Implementation Result

The custom application was successfully established as the foundation for the Employee Raise Issue solution. The application provides the development context for the custom Employee Issue table and the ServiceNow components used to implement the employee issue submission process.

The next implementation step is the creation and configuration of the **Employee Issue custom table**, which stores the structured information submitted through the Record Producer.

5.2 Employee Issue Custom Table

The **Employee Issue custom table** was created to store the structured information submitted by employees through the Raise Employee Issue Record Producer. The table acts as the primary data repository for the application and provides the structure required for managing submitted employee issues.

The custom table is created within the **Employee Center** application scope and uses the application's custom namespace.

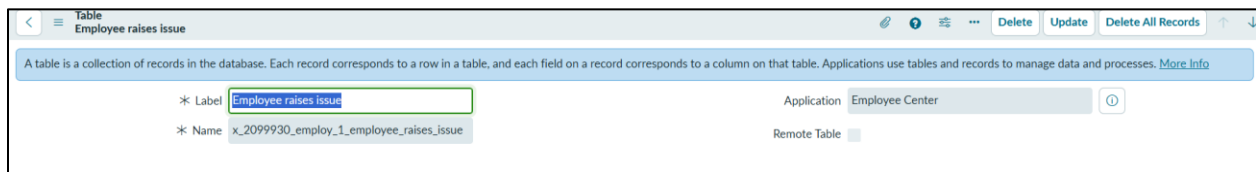
The screenshot shows the configuration page for a custom table named "Employee raises issue". At the top, there are tabs for "Table" and "Employee raises issue", and buttons for "Delete", "Update", and "Delete All Records". Below the tabs, a blue banner explains that a table is a collection of records. The configuration fields include: "Label" set to "Employee raises issue", "Name" set to "x_2099930_employ_1_employee_raises_issue", "Application" set to "Employee Center", and "Remote Table" set to "false".

Figure 6: Employee Issue Custom Table Configuration

The technical name of the table is:

x_2099930_employ_1_employee_raises_issue

The table contains fields corresponding to the information required for raising and managing an employee issue.

Table Fields

Field	Purpose
Number	Provides a unique identifier for each Employee Issue record.
Requester	Identifies the employee who raised the issue.
Category	Classifies the type of workplace-related issue.

Subcategory	Provides a more specific classification based on the selected Category.
Short Description	Captures a concise description of the employee's issue.
State	Represents the current state of the Employee Issue record.
Priority	Indicates the priority assigned to the issue.
Assignment Group	Identifies the group responsible for managing the issue.
Created	Stores the date and time when the record was created.
Created by	Identifies the user who created the record.
Sys ID	Stores the unique system identifier (GUID) of the record.
Updated	Stores the date and time when the record was last updated.
Updated by	Identifies the user who last updated the record.
Updates	Stores the number of times the record has been updated.

Columns Controls Application Access						
Table Columns Column label Search						
Dictionary Entries						
Column label	Type	Reference	Max length	Default value	Display	
✕ Assignment Group	Reference	Group		32	false	
✕ Category	String	(empty)	40		false	
Created	Date/Time	(empty)	40		false	
Created by	String	(empty)	40		false	
✕ Number	String	(empty)	40	javascript:global.getNextObjNumberPadded();	false	
✕ Priority	String	(empty)	40		false	
✕ Requester	Reference	User	32		false	
✕ Short Description	String	(empty)	40		false	
✕ State	String	(empty)	40		false	
✕ Subcategory	String	(empty)	40		false	
Sys ID	Sys ID (GUID)	(empty)	32		false	
Updated	Date/Time	(empty)	40		false	
Updated by	String	(empty)	40		false	
Updates	Integer	(empty)	40		false	
+ Insert a new row...						

Figure 7: Employee Issue Custom Table Fields

Table Form Configuration

The Employee Issue form provides a structured interface for viewing and managing the records created through the Record Producer. The form displays the key issue information, including the requester, category, subcategory, short description, state, priority, and assignment group.

Employee raises issue
EMP0001015

Number: EMP0001015

* Requester: Fred Luddy

Category: Hardware

Subcategory: Laptop

* Short Description: Test Case TC-006

State: New

Priority: Medium

Assignment Group: Hardware

Update Delete

[SN Utils | Versions \(0\)](#)

Figure 8: Employee Issue Record Form

The table therefore provides the persistent storage layer for the Employee Raise Issue solution. Once an employee submits the Record Producer, the submitted information is mapped to the corresponding table fields and a unique Employee Issue record is created.

Implementation Result

The Employee Issue custom table was successfully configured as the central data repository for the application. It provides structured storage for employee-raised issues and supports subsequent tracking, assignment, prioritization, and management of the created records.

The next step is to configure the **Raise Employee Issue Record Producer**, which provides the employee-facing mechanism for creating records in this table.

5.3 Raise Employee Issue Record Producer

The **Raise Employee Issue Record Producer** provides the employee-facing mechanism for submitting a workplace-related issue through the ServiceNow Service Portal. Instead of exposing the Employee Issue custom table directly to employees, the Record Producer provides a controlled and simplified submission interface.

The Record Producer is configured to create a new record in the **Employee raises issue** custom table when the employee submits the form.

The screenshot shows the 'Record Producer' configuration for 'Raise Employee Issue'. The interface includes a top navigation bar with buttons like 'Copy', 'Try It', 'Update', 'Edit in Catalog Builder', and 'Delete'. Below this is a header section with a message: 'Build and modify items faster with the improved Catalog Builder.' The main configuration area contains fields for 'Name' (Raise Employee Issue), 'Application' (Employee Center), 'Table name' (Employee raises issue [x_2099930_empl...]), 'State' (None), 'Checked out' (None), and 'Owner' (System Administrator). There are also tabs for 'What it will contain', 'Accessibility', 'Generated Record Data', and 'Portal Settings'. The 'What it will contain' tab is active, showing 'Catalogs' (Service Catalog) and 'Category' (Services). A 'Can cancel' checkbox is also present.

Figure 9: Raise Employee Issue Record Producer Configuration

Record Producer Configuration

The Record Producer was configured with the following purpose and target:

Configuration	Details
Record Producer	Raise Employee Issue
Application	Employee Center
Target Table	Employee raises issue
Submission Interface	Employee Center
Purpose	Capture employee issue information and create an Employee Issue record
Input Method	Record Producer variables

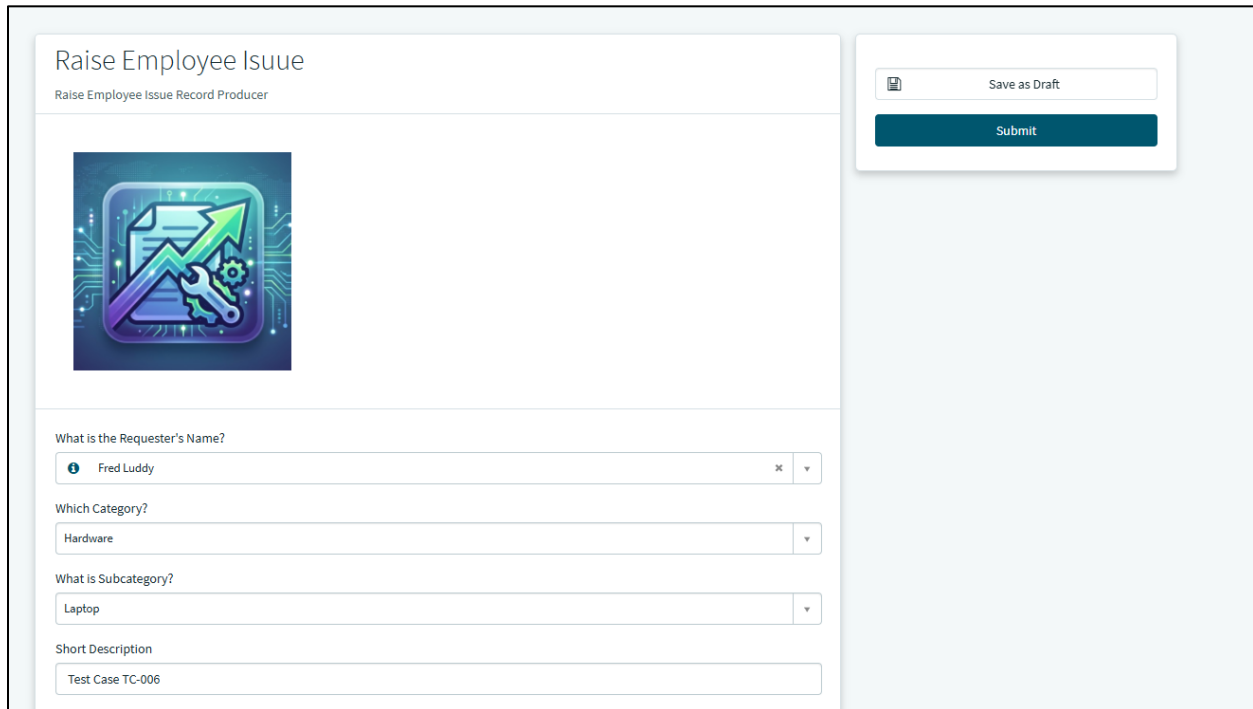
The Record Producer acts as the bridge between the employee-facing submission form and the Employee Issue custom table. The employee provides the required information through the form, while the Record Producer handles the creation of the corresponding table record.

Employee-Facing Submission Form

The Record Producer is exposed through the Employee Center so that employees can access the issue-raising functionality without navigating directly to the underlying custom table.

The submission form provides a structured interface for entering employee issue information. The configured variables include the requester, category, subcategory, state, and short description.

The employee enters the required information and selects **Submit**. The submitted values are then processed by the Record Producer and used to create the corresponding Employee Issue record.



Raise Employee Issue

Raise Employee Issue Record Producer

What is the Requester's Name?

Fred Luddy

Which Category?

Hardware

What is Subcategory?

Laptop

Short Description

Test Case TC-006

Save as Draft

Submit

Figure 10: *Raise Employee Issue Record Producer in the Service Portal*

Record Creation Flow

The Record Producer follows the following process:

Employee → Service Portal → Raise Employee Issue Record Producer → Record Producer Variables → Validation & Dependency Controls → Employee Issue Custom Table → Employee Issue Record

Implementation Result

The Raise Employee Issue Record Producer was configured successfully and connected to the Employee Issue custom table. Its variables capture the required issue information and map the submitted values to the corresponding table fields.

5.4 Record Producer Variables

The **Raise Employee Issue Record Producer** uses Record Producer variables to collect the information required from the employee during the issue submission process. These variables provide the input fields displayed on the employee-facing submission form.

The configured variables correspond to the information required for creating an **Employee Issue** record.

Question
What is the Requester's Name?
Which Category?
What is Subcategory?
Short Description
Which State is this issue in?

Figure 11: *Record Producer Variables for Raise Employee Issue*

Configured Variables

Variable	Purpose
Requester	Identifies the employee who is raising the issue.
Category	Captures the broad classification of the employee issue.
Subcategory	Captures the specific type of issue based on the selected Category.
State	Represents the state associated with the submitted issue.
Short Description	Captures a concise description of the issue being reported.

The variables provide a structured method for collecting employee issue information instead of requiring employees to interact directly with the fields of the underlying custom table.

The **Requester** variable allows an employee to be identified, while **Category** and **Subcategory** provide classification of the issue. The **Short Description** variable captures the primary description of the issue. The **State** variable represents the configured state information associated with the submission.

Variable-to-Field Relationship

The Record Producer variables are used as the input layer for the Employee Issue custom table. The information entered through the variables is processed by the Record Producer and mapped to the corresponding fields of the target table.

Question	Field
What is the Requester's Name?	u_requester
Which Category?	u_category
What is Subcategory?	u_subcategory
Short Description	u_short_description
Which State is this issue in?	u_state

Figure 12: Record Producer Variable-to-Field Mapping

The relationship can be represented as:

Employee Input → Record Producer Variables → Validation & Dependency Controls → Employee Issue Table Fields → Employee Issue Record

This variable-based approach separates the employee-facing submission interface from the underlying database structure and provides a controlled mechanism for collecting the required issue information.

Implementation Result

The Record Producer variables were successfully configured to capture the required employee issue information. These variables form the input layer of the **Raise Employee Issue** submission process and provide the data required for creating structured Employee Issue records.

5.5 Category and Subcategory Configuration

The **Category** and **Subcategory** fields provide a structured classification mechanism for Employee Issues. Category identifies the broad type of workplace-related issue, while Subcategory provides a more specific classification based on the selected Category.

This classification helps maintain consistent issue information and supports appropriate handling and management of submitted Employee Issues. The project requirements specifically include Category–Subcategory classification and a dependency in which Subcategory options respond to the selected Category.

Category Configuration

The **Category** variable is configured as a Select Box and is mapped to the `u_category` field of the Employee Issue custom table. Employees select the appropriate category while submitting an issue through the Raise Employee Issue Record Producer.

The screenshot displays the configuration interface for the 'Category' variable. The top section includes fields for 'Application' (Employee Center), 'Map to field' (checked), 'Type' (Select Box), 'Catalog item' (Raise Employee Issue), 'Order' (200), 'Field' (Category), and 'Record producer table' (x_2099930_employ_1_employee_raises_issue). On the right, there are checkboxes for 'Active' (checked), 'Mandatory', 'Read only', 'Hidden', and 'Disable automatic slot fill based on user context'. Below these fields is a tabbed interface with tabs for 'Question', 'Annotation', 'Type Specifications', 'Default Value', 'Auto-populate', 'Permission', and 'Availability'. The 'Type Specifications' tab is selected, showing a section titled 'Values specific to this variable Type'. This section includes a 'Variable Width' dropdown set to 'System Default Width (50) %', a 'Choice table' dropdown set to 'Employee raises issue [x_2099930_employ_1_employee_raises_issue]', a 'Choice field' dropdown set to 'Category', and checkboxes for 'Include none' and 'Unique values only'.

Figure 13: Employee Issue Category Configuration

Subcategory Configuration

The **Subcategory** variable is configured as a Select Box and is mapped to the `u_subcategory` field of the Employee Issue custom table. The available Subcategory values are controlled according to the selected Category.

Figure 14: Employee Issue Subcategory Configuration

Category–Subcategory Dependency

A Category–Subcategory dependency is implemented to ensure that employees select a Subcategory appropriate to the selected Category. When a Category is selected, the Subcategory options are controlled according to the configured dependency.

This prevents unrelated Subcategory values from being selected and helps maintain consistent issue classification.

Figure 15: Category–Subcategory Dependency Configuration

Implementation Result

The Category and Subcategory configuration provides a structured classification mechanism for Employee Issues. The dependency between the two fields ensures that Subcategory selection is controlled by the selected Category, improving consistency and reducing the possibility of incorrect classification.

The implemented behavior was also included in the project testing. **Test Case TC-003** verifies that the Subcategory options are displayed according to the selected Category, and the expected behavior was successfully observed.

5.6 UI Policies

UI Policies were configured to control field behavior dynamically on the Employee Issue table. The configuration ensures that system-generated fields remain protected and that required information is provided when an issue is in the **New** state.

5.6.1 Number Field Read-Only Policy

A UI Policy named “**Number field will read only.**” was created for the Employee Issue table.

The policy contains a UI Policy Action for the **Number** field with **Read only = True**. No condition is specified, so the Number field remains read-only on the form.

The screenshot shows the configuration page for a UI Policy named "Number field will read only." The interface includes a breadcrumb trail, a table selector (set to "Employee raises issue [x_2099930_empl...]"), an application selector (set to "Employee Center"), and an "Active" checkbox. A "Short description" field contains the text "Number field will read only." Below this is a "When to Apply" section with "Conditions" and "Add Filter Condition" / "Add OR Clause" buttons. A table of "UI Policy Actions (1)" is shown, with one action for the "number" field where "Read only" is set to "True".

Field name	Mandatory	Visible	Read only
number	Leave alone	Leave alone	True

Figure 16: *Number field will read only*

Purpose: Prevent users from manually modifying the system-generated Employee Issue number.

5.6.2 State Field Read-Only Policy

A UI Policy named “**State field will be read only.**” was created for the Employee Issue table.

The policy contains a UI Policy Action for the **State (u_state)** field with **Read only = True**. No condition is specified, so the State field remains read-only on the form.

Field name	Mandatory	Visible	Read only
u_state	Leave alone	Leave alone	True

Figure 17: *State field will be read only*

Purpose: Prevent users from directly modifying the issue state through the form.

5.6.3 Short Description Mandatory Policy

A third UI Policy named “**When the state changes to new then short description is mandatory.**” was configured.

The policy is applied when the following condition is satisfied:

State is New

A UI Policy Action is configured for the **Short Description (u_short_description)** field with **Mandatory = True**.

UI Policy

When the state changes to new then short description is mandatory.

Table: Employee raises issue [x_2099930_empl...]

Application: Employee Center

Active: ☒

* Short description: When the state changes to new then short description is mandatory.

When to Apply

Conditions: Add Filter Condition Add OR Clause

State is New

AND OR X

Update Delete

Related Links

[Convert this to Data Policy](#)

[Advanced view](#)

[Run Point Scan](#)

[\[SN Utils\] Versions \(1\)](#)

UI Policy Actions (1) UI Policy Related List Actions

for text Search

UI policy = When the state changes to new then short description is mandatory.

Field name	Mandatory	Visible	Read only
u_short_description	True	Leave alone	Leave alone

Figure 18: *When the state changes to new then Short Description is mandatory*

Purpose: Ensure that an issue in the New state contains a short description before it can be submitted or managed further.

5.6.4 Summary of UI Policies

UI Policy	Condition	Field / Action	Result
Number field will read only	None	Number → Read only	Number cannot be edited
State field will be read only	None	State → Read only	State cannot be edited
When the state changes to new then short description is mandatory	State is New	Short Description → Mandatory	Description is required when State is New

These UI Policies provide controlled field behavior and help maintain consistency and completeness of Employee Issue records.

5.7 Catalog Client Script

The **Catalog Client Script** was configured to implement the **Category–Subcategory dependency** in the *Raise Employee Issue* Record Producer. It controls the Subcategory options dynamically based on the Category selected by the employee.

5.7.1 Catalog Client Script

A **Catalog Client Script** named “**Filter Subcategory by Category**” was created for the **Raise Employee Issue** Record Producer.

The script executes **onChange** of the **Category** variable. When the Category changes, it clears the existing Subcategory value and removes the previously available Subcategory options. It then adds the appropriate Subcategory option based on the selected Category.

The implemented mapping is:

Category	Subcategory
Network	VPN
Hardware	Laptop
Software	Server
Access	Forgot Password

This ensures that the Subcategory selection remains consistent with the selected Category and prevents unrelated Subcategory options from being presented to the user.

The screenshot shows the configuration for a Catalog Client Script named "Filter Subcategory by Category". The script is associated with the "Employee Center" application and is triggered on the "onChange" event of the "Raise Employee Issue" catalog item, specifically for the "u_category" variable. The script is active and applies to all UI types. The script logic is as follows:

```
1 function onChange(control, oldValue, newValue, isLoading) {  
2  
3     if (!newValue) {  
4         g_form.clearValue('u_subcategory');  
5         g_form.clearOptions('u_subcategory');  
6         return;  
7     }  
8  
9     g_form.clearValue('u_subcategory');  
10    g_form.clearOptions('u_subcategory');  
11  
12    switch (newValue) {  
13  
14        case 'network':  
15            g_form.addOption('u_subcategory', 'vpn', 'VPN');  
16            break;  
17  
18        case 'hardware':  
19            g_form.addOption('u_subcategory', 'laptop', 'Laptop');  
20            break;  
21  
22        case 'software':  
23            g_form.addOption('u_subcategory', 'server', 'Server');
```

Figure 19: Filter Subcategory by Category Catalog Client Script Configuration

5.7.2 Script Logic

The implemented logic follows this sequence:

1. Check whether the script is executing during form loading.
2. If the form is loading, exit without applying the change logic.
3. Clear the currently selected **Subcategory** value.
4. Clear the existing Subcategory options.
5. Check the newly selected Category.
6. Add the appropriate Subcategory option based on the selected Category.

The configured Category–Subcategory mappings are:

Category	Subcategory
Network	VPN
Hardware	Laptop
Software	Server
Access	Forgot Password

This ensures that employees are presented only with the Subcategory appropriate to the selected Category.

5.7.3 Dependency Behavior

The dependency works dynamically on the employee-facing Record Producer form.

For example:

Category = Hardware

→ Subcategory options are filtered to **Laptop**

Category = Software

→ Subcategory options are filtered to **Server**

If the employee changes the Category, the previously selected Subcategory is cleared and the available options are updated according to the new Category. This prevents unrelated Subcategory values from being selected and maintains consistent classification of Employee Issues.

5.7.4 Initial-Load Handling

During testing, an initial-load issue was identified in which all Subcategory values were temporarily visible when the form initially loaded with **Network** selected. The filtering worked correctly after the Category was changed.

The Catalog Client Script was subsequently adjusted to handle the loading condition appropriately. After the adjustment, the dependency behaved as expected without initially exposing unrelated Subcategory values.

Implementation Result

The **Filter Subcategory by Category** Catalog Client Script was successfully configured to provide dynamic Category–Subcategory filtering in the Raise Employee Issue Record Producer. The script ensures that Subcategory values are controlled according to the selected Category, improving data consistency and reducing the possibility of incorrect issue classification.

5.8 Form Validation

The Employee Raise Issue submission process includes configured controls to ensure that the required issue information follows the defined application requirements.

The validation-related behavior is implemented through the configured **UI Policies** and **Catalog Client Script**. The UI Policies control field behavior on the Employee Issue form, while the Catalog Client Script controls the Category–Subcategory relationship on the Raise Employee Issue Record Producer.

The configured controls provide the following behavior:

- The **Short Description** field becomes mandatory when the issue State is **New**.
- The **Number** field remains read-only on the Employee Issue form.
- The **State** field remains read-only on the Employee Issue form.
- The **Subcategory** options are controlled according to the selected Category.
- Changing the Category clears the previously selected Subcategory and updates the available Subcategory option.

Validation Flow

The overall behavior can be represented as:

Employee Input → Record Producer Controls → Category–Subcategory Validation → Record Producer Processing → Employee Issue Record Creation

When the configured requirements are satisfied, the Record Producer can process the submission and create the corresponding Employee Issue record.

For example, during testing, leaving the Short Description empty while the State is New resulted in the submission being prevented, confirming the mandatory-field behavior.

Validation Result

The configured UI Policies and Catalog Client Script were tested as part of the Employee Raise Issue implementation. The UI Policies provide the required field behavior on the Employee Issue form, while the Catalog Client Script maintains the Category–Subcategory relationship on the Record Producer.

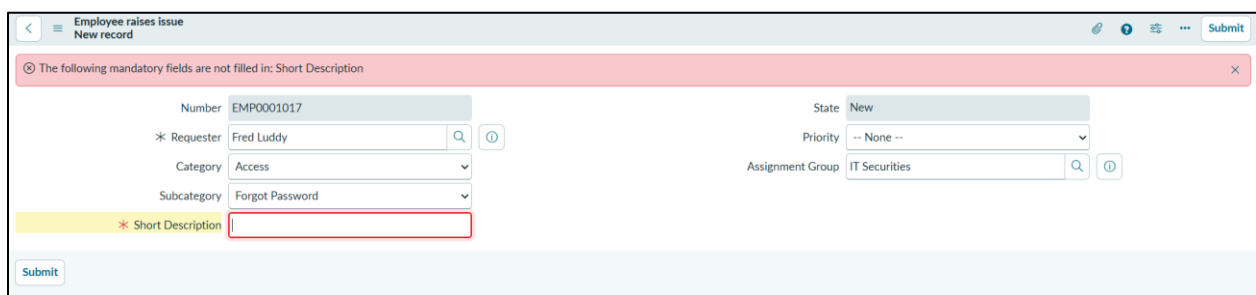
The screenshot shows a web form titled "Employee raises issue New record". At the top, a red error banner states: "The following mandatory fields are not filled in: Short Description". The form contains several fields: "Number" (EMP0001017), "Requester" (Fred Luddy), "Category" (Access), "Subcategory" (Forgot Password), "State" (New), "Priority" (-- None --), and "Assignment Group" (IT Securities). The "Short Description" field is highlighted with a red border and a red asterisk, indicating it is a mandatory field that has not been filled. A "Submit" button is located at the bottom left of the form.

Figure 20: *Employee Raise Issue Form Validation*

5.9 Employee Center

The **Employee Center** provides the employee-facing interface for accessing the issue-submission functionality. The application was configured so that employees can reach the **Raise Employee Issue** Record Producer through the portal using two different approaches.

5.9.1 Access through the Raise an Issue Widget

A custom **Raise an Issue** widget was provided on the Employee Center home page. The widget presents the employee with a dedicated **Raise Ticket** option for initiating an issue submission.

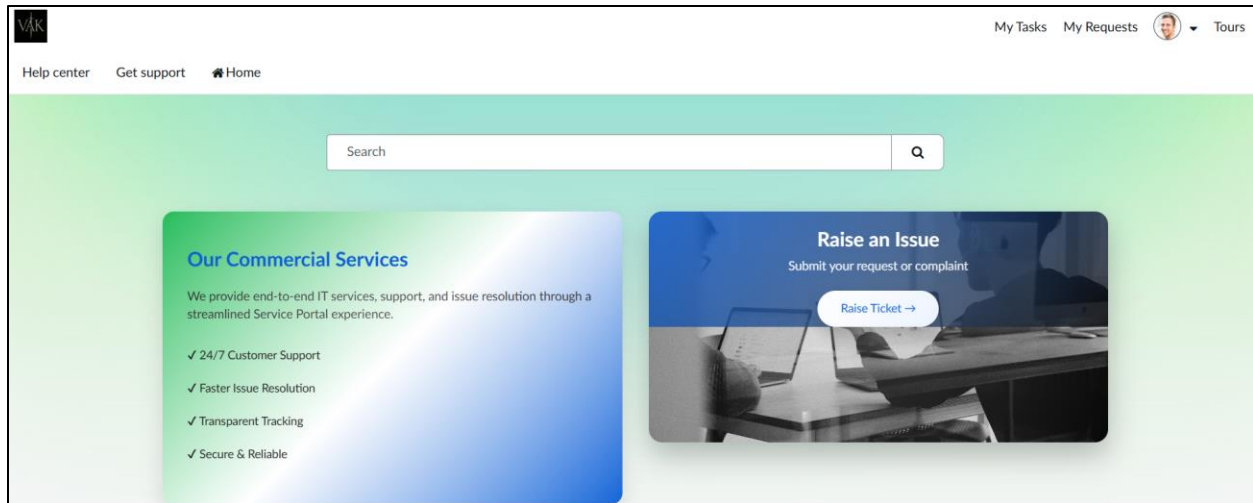


Figure 21: *Employee Center home page with the Raise an Issue widget*

Selecting **Raise Ticket** provides a direct route to the **Raise Employee Issue** Record Producer. This reduces the number of steps required for an employee to initiate an issue and provides a clear entry point for the service.

5.9.2 Access through Portal Search

The **Raise Employee Issue** Record Producer can also be accessed through the Employee Center search functionality. When the employee searches for “**Raise Employee Issue**”, the corresponding Record Producer appears in the search results.

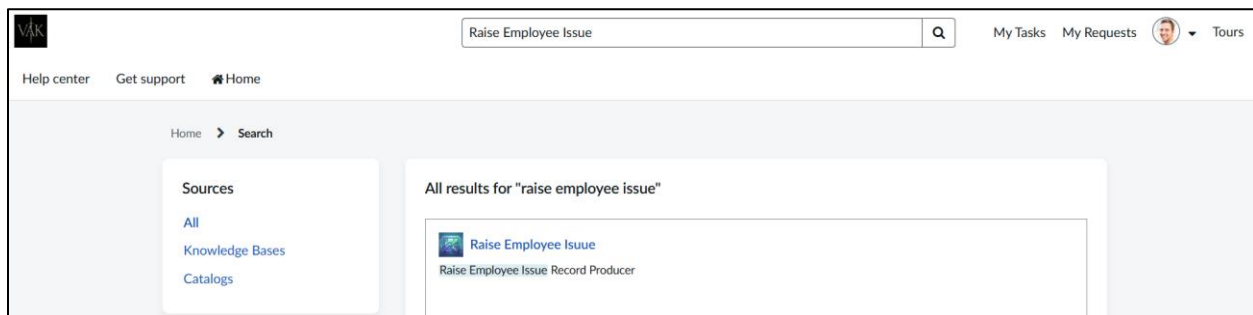


Figure 22: *Raise Employee Issue Record Producer displayed in Employee Center search results*

This provides an alternative access path for users who prefer to locate services through search rather than using the dedicated widget.

5.9.3 Employee Access Flow

The two access methods can therefore be represented as:

Employee Center Home Page → Raise an Issue Widget → Raise Ticket → Raise Employee Issue Record Producer

OR

Employee Center Search → “Raise Employee Issue” → Record Producer

Both approaches ultimately lead the employee to the same **Raise Employee Issue** Record Producer, where the configured variables, Category–Subcategory dependency, validation rules, and other application logic are applied.

5.10 Custom Widgets

The Employee Center home page includes a custom **Raise an Issue** widget that provides a direct entry point to the Employee Raise Issue submission process.

5.10.1 Raise an Issue Widget

The custom widget is displayed on the Employee Center home page and provides a dedicated **Raise Ticket** option for employees.

The purpose of the widget is to provide a clear and easily accessible entry point for raising an employee issue without requiring the employee to search for the Record Producer manually.

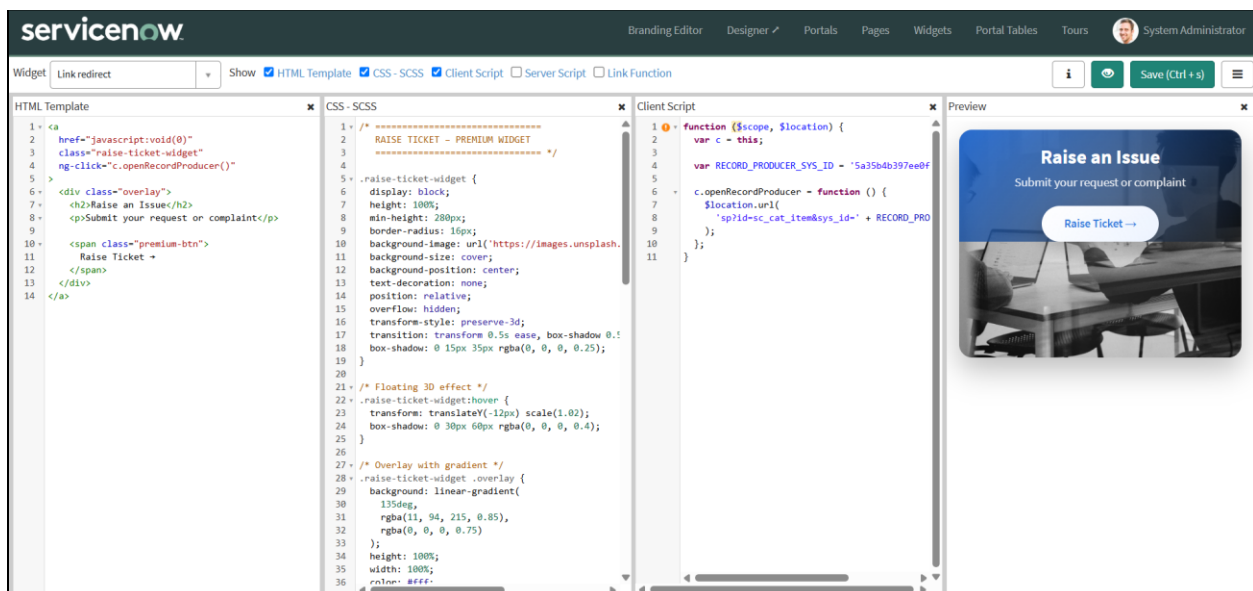


Figure 23: Raise an Issue Custom Widget on Employee Center

5.10.2 Widget Behavior

When the employee selects **Raise Ticket**, the widget directs the user to the **Raise Employee Issue Record Producer**.

The resulting flow is:

Employee Center → Raise an Issue Widget → Raise Ticket → Raise Employee Issue Record Producer

The Record Producer then presents the configured variables and applies the Category–Subcategory dependency and validation behavior described in the previous sections.

5.10.3 Widget Implementation Result

The Raise an Issue widget was successfully integrated into the Employee Center home page and provides a direct access path to the Employee Raise Issue submission process.

The widget improves accessibility by reducing the number of navigation steps required to initiate an issue.

5.11 End-to-End Workflow

The Employee Raise Issue application combines the configured ServiceNow components into a complete employee issue submission workflow.

The overall process begins when an employee accesses the Employee Center and ends when the submitted information is stored as an Employee Issue record.

5.11.1 Complete Workflow

The end-to-end workflow is:

Employee → Employee Center → Raise an Issue Widget / Portal Search → Raise Employee Issue Record Producer → Record Producer Variables → Validation & Dependency Controls → Employee Issue Custom Table → Employee Issue Record

The employee first accesses the Employee Center through either the dedicated Raise an Issue widget or the portal search functionality. Both access methods lead to the same Record Producer.

The employee then provides the required information through the configured Record Producer variables:

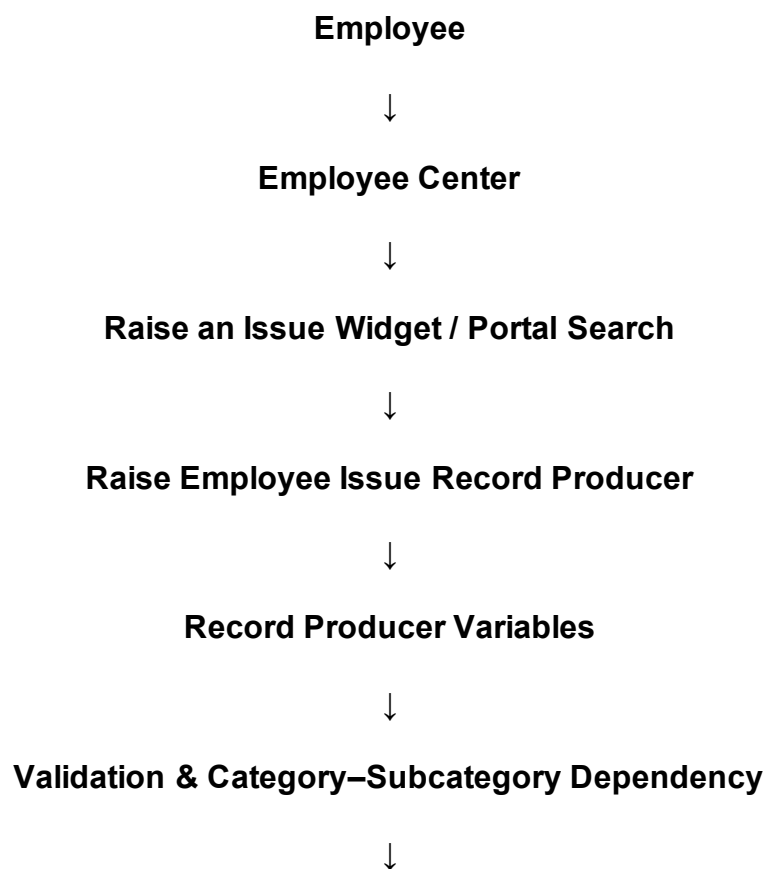
- Requester
- Category
- Subcategory
- State
- Short Description

The Category–Subcategory dependency controls the available Subcategory option according to the selected Category.

The configured UI Policies control the required and read-only behavior of the Employee Issue form.

After the required information is provided, the Record Producer processes the submission and creates the corresponding Employee Issue record in the custom table.

5.11.2 Workflow Representation



Employee Issue Custom Table



Employee Issue Record

5.11.3 Implementation Result

The complete workflow connects the employee-facing portal interface with the Record Producer, validation controls, custom table, and resulting Employee Issue record.

The implemented components therefore work together as a single issue-submission process.

5.12 Final Application Configuration

After completing the individual implementation components, the Employee Raise Issue application was reviewed as a complete ServiceNow solution.

5.12.1 Application Components

Component	Configuration
Application	Employee Center
Main Data Object	Employee Issue custom table
Table Name	x_2099930_employ_1_employee_raises_issue
Submission Mechanism	Raise Employee Issue Record Producer
Employee Interface	Employee Center
Input Layer	Record Producer Variables
Classification	Category and Subcategory
Dependency	Category–Subcategory filtering
Field Controls	UI Policies
Client-Side Logic	Catalog Client Script
Portal Access	Raise an Issue widget and portal search

The configuration described above is consistent with the application, table, Record Producer, variables, UI Policies, client-side dependency logic, and Employee Center functionality already documented in Sections 5.1–5.9.

5.12.2 Final Data Flow

The final application data flow is:

Employee Input → Record Producer Variables → Validation & Dependency Controls → Employee Issue Table Fields → Employee Issue Record

The implemented solution therefore provides a complete path from employee input to persistent Employee Issue record creation.

5.12.3 Final Implementation Result

The Employee Raise Issue application was successfully configured with the required ServiceNow components.

The final configuration provides:

- Employee-facing issue submission through Employee Center.
- Record Producer-based record creation.
- Structured issue information capture.
- Category–Subcategory dependency.
- Configured field validation and read-only behavior.
- Storage of submitted information in the Employee Issue custom table.
- Multiple access paths through the Employee Center.

6. TESTING AND OUTPUT

The implemented application was tested within the ServiceNow Personal Developer Instance to verify the configured submission, validation, dependency, portal access, and record-creation behavior.

6.1 Test Environment

Testing was performed in the same ServiceNow Personal Developer Instance used for application development.

The following components were included in testing:

- Employee Center
- Raise Employee Issue Record Producer
- Employee Issue custom table
- Record Producer variables
- UI Policies
- Category–Subcategory dependency
- Catalog Client Script

The PDI was used as the isolated development and testing environment for the project.

6.2 Test Scenarios

Test Scenario	Test Condition	Expected Result	Result
1	Access Raise Employee Issue through Employee Center widget	Record Producer should open	Passed
2	Search for Raise Employee Issue through portal search	Record Producer should appear in search results	Passed
3	Select a Category	Appropriate Subcategory should be displayed	Passed
4	Change the selected Category	Previous Subcategory should be cleared and updated	Passed
5	State is New and Short Description is empty	Submission should be prevented	Passed
6	Enter valid issue information and submit	Employee Issue record should be created	Passed
7	Open the created Employee Issue record	Submitted information should be available in the corresponding fields	Passed

The document specifically records successful Category–Subcategory testing as **TC-003**, and it also records that an empty Short Description while State was New prevented submission.

6.3 Output Screenshots

The major outputs of the completed application are represented by the screenshots included throughout the implementation section.

Employee Center Access

Figure 21 demonstrates the Employee Center home page containing the **Raise an Issue** widget and its **Raise Ticket** option.

Portal Search

Figure 22 demonstrates that searching for **Raise Employee Issue** displays the corresponding Record Producer in the Employee Center search results.

Record Producer

Figure 10 demonstrates the employee-facing Raise Employee Issue submission form with the configured input variables.

Validation

Figure 20 demonstrates the mandatory Short Description validation when the State is New.

Created Employee Issue Record

Figure 8 demonstrates the resulting Employee Issue record containing the submitted issue information.

These outputs collectively demonstrate the implemented employee access, submission, validation, and record-management flow.

7. CONCLUSION

The **Employee Raise Issue Management** application was successfully implemented using ServiceNow platform capabilities.

The project provides employees with a centralized mechanism for submitting workplace-related issues through the Employee Center. A Record Producer was configured to capture the required issue information and create corresponding records in the Employee Issue custom table.

The application also implements Category–Subcategory dependency, UI Policies, Catalog Client Script logic, validation behavior, and multiple portal access methods.

The final workflow connects the employee-facing portal interface with the Record Producer, validation controls, custom table, and resulting Employee Issue record.

Overall, the project demonstrates how ServiceNow can be used to develop a structured employee-service application by combining **custom tables, Record Producers, variables, UI Policies, Catalog Client Scripts, and Employee Center integration**.

The implemented solution provides a foundation that can be extended with additional functionality such as assignment, notifications, workflow automation, reporting, and further employee-service capabilities.